Chapter 31: Signals vs RxJS

1. Overview

Angular now ships two reactive primitives that look superficially similar — both let you react to changing data — but they come from fundamentally different computational models. **Signals** are a synchronous, pull-based, glitch-free primitive for representing a single current value that changes over time. **RxJS Observables** are a push-based, asynchronous-capable primitive for representing a sequence of values (zero, one, many, or infinite) distributed over time, with a rich algebra of operators for composing, filtering, timing, and canceling those sequences.

The practical consequence: signals are the right tool for *derived UI state* — the kind of thing that used to live in `getters`, `ngOnChanges`, or ad-hoc `BehaviorSubject` plumbing just to make a template re-render. RxJS is the right tool for *process orchestration* — coordinating multiple async sources, debouncing user input, retrying failed HTTP calls, canceling in-flight requests, or merging streams from different origins with precise timing control.

This chapter builds a rigorous mental model of the two systems, shows where each shines and where each struggles, and covers the official interop layer (`@angular/core/rxjs-interop`) that lets you cross between them without building fragile bridges by hand. Angular does not want you to pick one and abandon the other — it wants you to know which one owns which concern.

---

2. Core Concepts

2.1 The Fundamental Model Difference

Dimension	Signals	RxJS Observables
Execution model	Pull-based (read on demand) with push-based *invalidation* (dependents are notified something changed, then re-read lazily)	Push-based (producer emits, consumers react immediately)
Synchronicity	Always synchronous — reading a signal returns a value *now*, no microtask/macrotask involved	Can be sync or async; many operators (`debounceTime`, `delay`, `switchMap` to HTTP) are inherently async
Cardinality	Exactly one current value at all times (never "no value", never "many values")	Zero, one, many, or infinite values over the observable's lifetime
Glitch-freedom	Guaranteed — Angular's signal graph uses a dependency-tracking + dirty-marking algorithm so a `computed()` is never read in a transiently inconsistent state, even with diamond dependencies	Not guaranteed — combining streams (`combineLatest`, `withLatestFrom`) can produce transient/inconsistent intermediate emissions unless carefully managed

Dimension	Signals	RxJS Observables
Consumption	`computed()` for derived synchronous values, `effect()` for side effects, direct call `signal()` for reads	`subscribe()`, operators (`map`, `filter`, `switchMap`, ...), async pipe in templates
Laziness	`computed()` is lazy and memoized — recomputed only when read *and* stale	Cold observables are lazy per-subscription (re-executed for each subscriber unless multicast); hot observables (`Subject`) are eager/always-running
Error semantics	No built-in error channel — an exception thrown while computing a `computed()` propagates synchronously to the reader; there's no "errored state" that subsequent reads observe distinctly from a thrown value each time	First-class `error` channel — a stream can terminate with an error, propagated once to `subscribe({error})`, after which the observable is dead
Completion semantics	No concept of "completion" — a signal is alive as long as it's referenced; it doesn't "finish"	First-class `complete` channel — a stream can finish, notifying subscribers, and stops emitting afterward
Cancellation	Not applicable — there's no in-flight "operation" to cancel, just a value to stop reading	Central feature — unsubscribing tears down the entire pipeline, cancels HTTP requests (via `HttpClient`'s use of the subscription lifecycle), clears timers
Composition primitives	`computed()` (derive), `effect()` (react/side-effect), `linkedSignal()` (writable derived state that can be reset)	Dozens of operators: `switchMap`, `mergeMap`, `debounceTime`, `retry`, `catchError`, `combineLatest`, `scan`, etc.
Multicasting	N/A — every consumer reading a signal sees the same current value trivially, no subject needed	Needs explicit multicasting (`share()`, `shareReplay()`, `Subject`) to avoid re-executing cold producers per subscriber
Angular integration	Native — signals directly drive change detection (zoneless-ready), template bindings read signals without `async` pipe	Requires `| async` pipe (auto-subscribe/unsubscribe) or manual subscription management

Dimension	Signals	RxJS Observables
Memory/teardown	Automatic — a signal has no subscription to leak; `effect()` inside a component is auto-destroyed with the component's injector	Manual or pipe-based — forgetting to unsubscribe is a classic memory leak source; `takeUntilDestroyed()` mitigates this
Backpressure/timing operators	None — no `debounceTime`, `throttleTime`, `auditTime` equivalents	Rich set of timing operators built exactly for this
Testability	Trivial — read the signal, assert the value, no `fakeAsync` / `tick()` needed for sync signals	Often needs `fakeAsync`, `TestScheduler`, or marble testing for time-based operators

2.2 Pull vs Push, Precisely

- **Push (RxJS):** the producer decides when data is delivered. `interval(1000)` pushes a value every second regardless of whether anyone is "asking." Subscribers are passive recipients.
- **Pull (Signals):** the *consumer* decides when to read. `mySignal()` just returns the current value synchronously at the call site. Nothing is "pushed" to you — you ask, and you get the latest snapshot right now. What signals do push is *invalidation*: when a dependency changes, dependents are marked dirty so the next pull recomputes. The value itself is never pushed through the graph; only a "you might be stale" notification is.

This is why `computed()` is cheap to sprinkle everywhere — it does no work until read, and Angular's dependency tracker (an implicit tracking context similar to MobX/Vue's reactivity, using a global "current computation" pointer during evaluation) records exactly which signals were read during the computation, building the dependency graph automatically without explicit subscriptions.

2.3 Glitch-Freedom

Consider:

```
a = signal(1)
b = computed(() => a() * 2)
c = computed(() => a() + b())
```

If `a` changes from 1 to 2, a naive push-based propagation could recompute `c` once when `a` changes (seeing new `a`, old `b`) and again when `b` finishes recomputing (seeing new `a`, new `b`) — a transient "glitch" with an inconsistent intermediate value. Angular's signal implementation avoids this: it marks the whole downstream graph dirty first (without recomputing), and only recomputes values lazily on read, always fully resolving dependencies bottom-up before a dependent is evaluated. The result: `c` is *never* observed in an inconsistent state. RxJS's `combineLatest` / `withLatestFrom` have no such guarantee — you can and do get multiple emissions during "settling" of dependent streams unless you architect around it (e.g., with `distinctUntilChanged`, careful operator ordering, or debouncing).

2.4 When To Use Which — Decision Framework

Use **signals** when:

- The value is derived synchronously from other in-memory state (component/service state).
- You want automatic, fine-grained change detection with no manual subscription lifecycle.
- There's no timing, cancellation, retries, or merging-of-multiple-async-sources concern.
- Example: `fullName = computed(() => ${first()} ${last()})`, `isFormValid = computed(...)`, `filteredList = computed(() => list().filter(...))`.

Use **RxJS** when:

- You're orchestrating asynchronous operations: HTTP calls, WebSocket streams, timers.
- You need cancellation semantics (`switchMap` canceling a stale in-flight request).
- You need timing operators: `debounceTime`, `throttleTime`, `auditTime`, `delay`, `bufferTime`.
- You need to combine multiple independent async sources with precise merge strategies (`merge`, `concat`, `combineLatest`, `zip`, `forkJoin`).
- You need retry/backoff logic (`retry`, `retryWhen`), or error recovery pipelines (`catchError`).
- You need a stream that can literally end (`complete`) and have subscribers react to that specific event.

Use **both together** (interop) when a component needs a signal-driven template but the underlying data source is push/async — e.g., an HTTP-backed resource or a WebSocket feed that should be exposed to the template as a plain signal.

---

3. Code Examples

3.1 `toSignal()` — Observable → Signal

```
import { Component, inject } from '@angular/core';
import { toSignal } from '@angular/core/rxjs-interop';
import { HttpClient } from '@angular/common/http';
import { interval } from 'rxjs';
import { map } from 'rxjs/operators';

@Component({
  selector: 'app-clock',
  template: `
    <p>Seconds elapsed: {{ elapsed() }}</p>
    <p>User: {{ user()?.name ?? 'loading...' }}</p>
  `,
})
export class ClockComponent {
  private http = inject(HttpClient);

  // Requires an initial value OR use requireSync/initialValue option.
  elapsed = toSignal(interval(1000).pipe(map(n => n + 1)), { initialValue: 0 });

  // HTTP observable -> signal; no initial value means signal starts as `undefined`
```

```
  user = toSignal(this.http.get<{ name: string }>('/api/user'));
}
```

Key points embedded in this example:

- `toSignal` must run in an injection context (constructor, field initializer, or inside `runInInjectionContext`) because it needs to grab the current `DestroyRef` / `Injector` to know when to unsubscribe.
- Without `{ initialValue: ... }`, the returned signal's type includes `undefined` until the first emission arrives (or you pass `{ requireSync: true }` to assert the observable emits synchronously on subscribe, which throws at runtime if it does not).

3.2 `toObservable()` — Signal → Observable

```
import { Component, signal, inject, Injector } from '@angular/core';
import { toObservable } from '@angular/core/rxjs-interop';
import { switchMap, debounceTime, distinctUntilChanged } from 'rxjs/operators';
import { HttpClient } from '@angular/common/http';

@Component({
  selector: 'app-search-bridge',
  template: `<input (input)="query.set($any($event.target).value)" />`,
})
export class SearchBridgeComponent {
  private http = inject(HttpClient);
  query = signal('');

  // Convert the signal to an observable so we can use RxJS timing/orchestration operators
  results$ = toObservable(this.query).pipe(
    debounceTime(300),
    distinctUntilChanged(),
    switchMap(q => q ? this.http.get(`/api/search?q=${q}`) : []),
  );
}
```

This is the idiomatic bridge: signals own the *state* (what the user typed), RxJS owns the *process* (debounce, cancel-and-restart via `switchMap`, HTTP call). `results$` is then consumed in the template with `| async`, or converted back with `toSignal(results$, { initialValue: [] })` if you want a pure-signal template.

3.3 Typeahead Search — RxJS Version (the natural fit)

```
import { Component, inject } from '@angular/core';
import { FormControl, ReactiveFormsModule } from '@angular/forms';
import { HttpClient } from '@angular/common/http';
import { toSignal } from '@angular/core/rxjs-interop';
import {
  debounceTime, distinctUntilChanged, switchMap,
  catchError, filter,
} from 'rxjs/operators';
import { of } from 'rxjs';
```

```
@Component({
  selector: 'app-typeahead-rxjs',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <input [formControl]="searchCtrl" />
    <ul>
      <li *ngFor="let item of results()">{{ item }}</li>
    </ul>
  `,
})
export class TypeaheadRxjsComponent {
  private http = inject(HttpClient);
  searchCtrl = new FormControl('', { nonNullable: true });

  results = toSignal(
    this.searchCtrl.valueChanges.pipe(
      filter(v => v.length >= 2),
      debounceTime(300),
      distinctUntilChanged(),
      switchMap(term =>
        this.http.get<string[]>(`/api/search?q=${term}`).pipe(
          catchError(() => of([] as string[])),
        ),
      ),
    ),
    { initialValue: [] as string[] },
  );
}
```

Everything a typeahead needs falls out of built-in operators: `debounceTime` (wait for typing to pause), `distinctUntilChanged` (skip redundant identical queries), `switchMap` (cancel the previous in-flight HTTP call the instant a new keystroke qualifies), `catchError` (recover per-request without killing the whole stream). This is exactly the scenario RxJS was designed for.

3.4 Typeahead Search — Attempted With Signals + `effect()` (where signals struggle)

```
import { Component, signal, effect, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Component({
  selector: 'app-typeahead-signals',
  template: `
    <input (input)="query.set($any($event.target).value)" />
    <ul><li *ngFor="let item of results()">{{ item }}</li></ul>
  `,
})
export class TypeaheadSignalsComponent {
  private http = inject(HttpClient);
```

```
  query = signal('');
  results = signal<string[]>([]);

  private debounceHandle?: ReturnType<typeof setTimeout>;
  private currentRequestToken = 0;

  constructor() {
    effect((onCleanup) => {
      const term = this.query();

      // Manually reimplementing debounceTime with setTimeout:
      clearTimeout(this.debounceHandle);
      if (term.length < 2) {
        this.results.set([]);
        return;
      }

      this.debounceHandle = setTimeout(() => {
        // Manually reimplementing switchMap's cancellation with a token,
        // because fetch has no built-in "cancel the stale one" semantics tied
        // to signal re-evaluation.
        const myToken = ++this.currentRequestToken;
        const controller = new AbortController();

        this.http // NOTE: HttpClient returns an Observable; using fetch here
                  // to show the "no RxJS at all" version conceptually.
          ;

        fetch(`/api/search?q=${term}`, { signal: controller.signal })
          .then(r => r.json())
          .then((data: string[]) => {
            if (myToken === this.currentRequestToken) {
              this.results.set(data);
            }
          })
          .catch(() => {
            if (myToken === this.currentRequestToken) this.results.set([]);
          });

        onCleanup(() => controller.abort());
      }, 300);

      onCleanup(() => clearTimeout(this.debounceHandle));
    });
  }
}
```

Notice everything that had to be hand-rolled: a manual `setTimeout` to fake `debounceTime`, a manual monotonically-increasing token (or `AbortController`) to fake `switchMap`'s cancel-the-stale-request behavior, manual `onCleanup` wiring for both the timer and the abort controller, and no composable `catchError`/`retry` story. This is strictly more code, more edge cases (what if `onCleanup` runs after `results.set` already happened in a race?), and it reinvents primitives RxJS gives you for free. **This is the**

canonical example of "signals win at derived state, RxJS wins at async orchestration." Don't build this in production — it's shown here specifically to illustrate *why* you reach for RxJS instead.

3.5 Simple Derived UI State — Signals Are Clearly Superior

```
import { Component, signal, computed } from '@angular/core';

@Component({
  selector: 'app-cart-summary',
  template: `
    <p>Items: {{ itemCount() }}</p>
    <p>Subtotal: {{ subtotal() | currency }}</p>
    <p>Tax: {{ tax() | currency }}</p>
    <p>Total: {{ total() | currency }}</p>
    <p *ngIf="isEligibleForFreeShipping()">You qualify for free shipping!</p>
  `,
})
export class CartSummaryComponent {
  items = signal<{ price: number; qty: number }[]>([]);

  itemCount = computed(() => this.items().reduce((n, i) => n + i.qty, 0));
  subtotal = computed(() => this.items().reduce((sum, i) => sum + i.price * i.qty, 0));
  tax = computed(() => this.subtotal() * 0.0825);
  total = computed(() => this.subtotal() + this.tax());
  isEligibleForFreeShipping = computed(() => this.subtotal() >= 50);
}
```

No subscriptions, no `async` pipe, no `BehaviorSubject` + `combineLatest` + manual `map` chains, no teardown to manage, no glitch risk between `subtotal` / `tax` / `total` even though `total` transitively depends on `subtotal` twice (once directly, once via `tax`). This is trivially testable: `component.items.set([...]); expect(component.total()).toBe(...)` — fully synchronous, no `fakeAsync` / `tick()` required. Compare to the old idiomatic RxJS-only equivalent, which needed a `BehaviorSubject<Item[]>`, several `.pipe(map(...))` derivations, and `| async` everywhere in the template just to get equivalent behavior — strictly more ceremony for a purely synchronous derivation.

---

4. Internal Working

4.1 How `toSignal()` Subscribes and Tears Down

`toSignal(source, options)` internally:

1. **Captures the injection context.** It calls `assertInInjectionContext(toSignal)` (unless you pass `{ injector }` explicitly) and retrieves the current `Injector`. From that injector it resolves a `DestroyRef`.
2. **Creates a `WritableSignal`** seeded with `options.initialValue` if provided (or left in an "undefined"/not-yet-emitted internal state that throws if read too early when `requireSync: true` and nothing has emitted synchronously).
3. **Subscribes to the source observable immediately** (eagerly, at `toSignal()` call time — not lazily on

first read). On each `next(value)`, it calls the internal writable signal's `.set(value)`, which triggers Angular's normal dirty-marking/notification for any `computed()` / `effect()` depending on it.

4. **Registers teardown via `DestroyRef.onDestroy(() => subscription.unsubscribe())`.** This ties the observable subscription's lifetime to the *injector's* lifetime — typically the component or the providing service — not to any Angular-specific "signal destroy" concept, since signals themselves have no destroy hook. This is exactly why an injection context is mandatory: without a `DestroyRef` to hook, the subscription would leak for the lifetime of the app.
5. On `error`, `toSignal` by default lets the error propagate: reading the signal after the source errors re-throws (unless you pass a `manualCleanup` / `equal` /custom error-handling strategy such as wrapping the observable with `catchError` yourself before passing it in — `toSignal` itself does not swallow errors silently).
6. On `complete`, the last emitted value remains the signal's steady-state value; no further updates occur (there's no representation of "completed" state — it just stops changing, same as any other signal that nobody calls `.set()` on anymore).

4.2 How `toObservable()` Creates an Observable From a Signal

`toObservable(sourceSignal, options)` internally:

1. Creates an RxJS observable using a constructor-style approach (conceptually similar to wrapping in `new Observable(subscriber => {...})`).
2. Inside, on subscription, it sets up an `effect()` (using the injection context captured the same way as `toSignal` — either ambient or an explicit `{ injector }`) whose body simply reads `sourceSignal()` and calls `subscriber.next(value)`.
3. Because `effect()` runs **once synchronously on creation** (to establish its dependencies), the observable emits the signal's *current* value immediately upon subscription — `toObservable` therefore behaves like a `BehaviorSubject` wrapper: subscribers always get the current value right away, then subsequent values on change.
4. Subsequent changes to the signal don't push synchronously *through* the effect the instant `.set()` is called; the effect is scheduled to run during Angular's next change-detection/effect-flush cycle (effects run as a scheduled reaction, batched via the microtask-based effect scheduler), not necessarily in the same synchronous stack frame as the `.set()` call. This means `toObservable` emissions are effectively async relative to the `.set()` call site, even though signal reads themselves are synchronous — a subtlety worth internalizing (see Edge Cases below).
5. Teardown: when the subscriber unsubscribes, the internal `effect()` is destroyed (its cleanup function is invoked), stopping further emissions. If the injector context that created the effect is itself destroyed, the effect also tears down, closing the observable from that side too.

---

5. Edge Cases & Gotchas

1. `toSignal()` **requires an injection context (or explicit injector).** Calling it inside a plain method, a `setTimeout` callback, or after `async` / `await` without capturing the injector first throws `NG0203` (`toSignal() can only be used within an injection context`). Fix: pass `{ injector: this.injector }` where `injector = inject(Injector)` was captured earlier, or wrap the call in `runInInjectionContext(injector, () => toSignal(...))`.

2. **No initial value means `undefined` is baked into the type**, and templates/consumers must null-check (`user()?.name`). Forgetting this is a common source of `Cannot read property 'name' of undefined` bugs right after component init, before the first HTTP response lands. `{ requireSync: true }` avoids the `undefined` type but throws at *runtime* if the source doesn't emit synchronously on subscribe (e.g., a `BehaviorSubject` qualifies, a plain `HttpClient.get()` does not).
3. `toObservable()` **emissions lag behind** `.set()` **by a scheduler tick.** Because it's effect-based, code like:

   ```
   mySignal.set(5);
   // subscriber's next(5) has NOT necessarily run synchronously here —
   // it's scheduled, not immediate.
   ```

   can surprise developers expecting RxJS-style synchronous `Subject.next()` semantics. If you rely on `toObservable` output inside the *same* synchronous block that mutated the signal, you may read stale data. This rarely matters in template-driven flows (change detection settles before render) but bites in tests that don't flush microtasks/effects (use `TestBed.flushEffects()` or `await` a tick).
4. **Multiple rapid** `.set()` **calls between effect flushes are coalesced.** If a signal is set to `1`, then `2`, then `3` synchronously before Angular flushes effects, `toObservable` will typically only emit the final settled value (`3`), not all three intermediate values — unlike a `Subject.next()` sequence, which would emit `1, 2, 3` individually. This is a real semantic difference: signals model *current value*, not *event history*, so intermediate transient values are not observable through this bridge.
5. **Overusing** `effect()` **to fake RxJS orchestration is an anti-pattern.** As shown in section 3.4, replicating `debounceTime` / `switchMap` / `retry` via `effect()` + manual timers/tokens/ `AbortController` produces more code, more race-condition surface area, and loses composability (you can't `.pipe()` more operators onto a hand-rolled effect chain). If you catch yourself writing `clearTimeout` or a "cancellation token" counter inside an `effect()`, that's a strong signal (pun intended) you should convert to `toObservable()` and finish the job in RxJS.
6. `effect()` **cannot be used to synchronously produce a new signal value inside itself in a naive way** — Angular disallows writing to signals that the effect also reads (to prevent infinite loops) unless you explicitly opt in via `allowSignalWrites` (deprecated in newer versions in favor of restructuring with `computed()`). This pushes you toward `computed()` for derivations and reserves `effect()` for genuine side effects (logging, DOM manipulation, syncing to non-Angular state) — reinforcing that `effect()` is not a general-purpose replacement for RxJS operator chains.
7. **Error handling asymmetry.** An RxJS stream that errors is *dead* — no more emissions, `error` callback fires once. If you `toSignal()` a stream that later errors without a `catchError` upstream, the *signal* doesn't have a clean "errored" state to render — the error re-throws when the signal is next read (inside a `computed()` or template binding), potentially crashing an unrelated render. Always pipe `catchError` before `toSignal()` if failure is expected and recoverable.
8. `toSignal()` **inside a** `computed()` **is not supported/meaningful** — `toSignal` performs a subscription side effect and must run once in a stable injection context, not inside a `computed()`'s repeatedly-re-evaluated pure function. Only call it at field-initialization or constructor time.
9. **Testing timing mismatch.** Tests that mix `TestBed`, signals, and observables sometimes need both `fakeAsync` / `tick()` (for the RxJS side, e.g. `debounceTime`) *and* an explicit change-detection/effect flush (for the signal side) in the same test, which can be non-obvious the first time you write it.

6. Interview Questions & Answers

Q1. In one sentence, what's the core model difference between a signal and an observable?
A signal represents a single current value you *pull* synchronously on demand, with automatic dependency tracking and glitch-free propagation; an observable represents a *pushed* sequence of zero-to-many values over time, with explicit subscription, operators, and first-class error/completion/cancellation semantics.

Q2. Can a signal have multiple values "in flight" the way an observable can emit a burst of values?
No. A signal always holds exactly one current value. Rapid successive `.set()` calls before anything reads the signal simply overwrite each other — there is no queue or history. An observable, by contrast, can emit and deliver every discrete value in a burst to its subscribers.

Q3. Why does `toSignal()` require an injection context?
Because it needs a `DestroyRef` to register automatic unsubscription — it ties the underlying subscription's lifetime to the enclosing component/service/directive's destruction. Without an injector, there's no way to know when to tear down the subscription, and it would leak.
Interviewer intent: checks whether the candidate understands that `toSignal` isn't magic — it's a real `Subscription` under the hood that needs a teardown hook, and that hook comes from Angular's DI/lifecycle system, not from the signal system itself.

Q4. What happens if you call `toSignal()` on an observable with no `initialValue` and read the signal before the first emission?
The signal returns `undefined` (its type becomes `T | undefined`), unless you pass `{ requireSync: true }`, in which case Angular throws at runtime if the observable didn't emit synchronously upon subscription. There is no "loading" state built in — you model that yourself (e.g., check for `undefined`, or seed a sentinel `initialValue`).

Q5. Why is `computed()` described as "glitch-free" and why does that matter?
Because Angular's signal graph marks all transitively-dependent computed signals dirty *before* recomputing any of them, and only recomputes lazily on read, fully resolving dependencies bottom-up. This guarantees a `computed()` is never read in a state that reflects only *some* of its dependencies having updated. It matters because naive push-based combination of multiple streams (e.g., `combineLatest`) can produce exactly such an inconsistent intermediate emission, which can cause subtle UI bugs (e.g., total shown before tax updates).
Interviewer intent: distinguishes candidates who've only used signals superficially from those who understand *why* the signals implementation was engineered the way it was — this is a common "explain the internals" trap question.

Q6. When would you deliberately choose RxJS over signals for what looks like "just derived state"?
When the derivation isn't purely synchronous — e.g., it depends on debounced user input, needs to cancel an in-flight async computation when new input arrives, needs to combine multiple async sources with specific timing/ordering guarantees, or needs retry/backoff on failure. Signals have no concept of timing operators, cancellation, or multi-value sequences, so any of these requirements pulls you toward RxJS (possibly bridging back to a signal at the boundary with `toSignal()`).

Q7. Walk through what `toObservable()` does internally.
It creates an observable that, on subscription, sets up an `effect()` inside the captured injection context. That effect's body reads the source signal and calls `subscriber.next()` with the value. Because `effect()` runs once immediately to track dependencies, the observable emits the current value right away (BehaviorSubject-like behavior), and then emits again whenever the effect re-runs due to a dependency change — which

happens on Angular's scheduled effect-flush, not necessarily synchronously with the `.set()` call. Unsubscribing destroys the underlying effect.

Q8. Why can `toObservable()` "miss" intermediate values if a signal is set multiple times quickly?
Because effects are scheduled/batched, not run synchronously on every `.set()`. If the signal changes from 1 → 2 → 3 all before the next effect-flush cycle, the effect only sees the final settled value (3) once it runs, so the observable emits `3`, not `1, 2, 3`. This is a real semantic difference from `Subject.next()`, which delivers every discrete emission synchronously to subscribers.
Interviewer intent: probes whether the candidate has actually used `toObservable()` in a nontrivial scenario (e.g., feeding a signal into a `switchMap` chain) and hit this surprise, versus just knowing the API exists.

Q9. How would you implement a debounced search box: with signals only, with RxJS, or a hybrid? Justify your choice.
Pure signals would require hand-rolling `setTimeout`-based debouncing and a manual cancellation token for stale requests inside an `effect()` — more code, more race conditions, no composability. Pure RxJS (`valueChanges.pipe(debounceTime(300), distinctUntilChanged(), switchMap(...))`) is the natural fit — those are exactly the operators RxJS was built for. The idiomatic Angular-signals-era answer is a **hybrid**: keep the input value in a `signal()`, bridge it to an observable with `toObservable()`, apply `debounceTime` / `switchMap` there, and bridge the result back to a `signal()` with `toSignal()` so the template stays signal-driven end to end.

Q10. Does an `effect()` provide the same cancellation guarantee as `switchMap`?
No, not out of the box. `switchMap` automatically unsubscribes from (and, for `HttpClient`, cancels) the previous inner observable the instant a new source value arrives. An `effect()` gives you an `onCleanup()` callback that runs before the next execution or on destroy, but you must manually wire that cleanup to cancel whatever async work you started (e.g., call `AbortController.abort()` yourself) — Angular does not do this automatically for arbitrary async work inside an effect.

Q11. What's wrong with this code?

```
effect(() => {
  this.count.set(this.count() + 1);
});
```

This effect reads and writes the same signal it depends on, which would cause infinite re-triggering; Angular disallows signal writes from within an effect that also reads that signal within the same execution unless explicitly using an escape hatch, and even then it's discouraged. The fix is almost always to model this as a `computed()` if it's a pure derivation, or to move the mutation to an explicit event handler (e.g., a button click) rather than an automatically-triggered effect.
Interviewer intent: tests whether the candidate understands effects are for side effects, not for general reactive computation, and knows the loop-prevention guard exists.

Q12. Explain the error/completion semantics gap between signals and observables, and why it matters for `toSignal()`.
Observables have first-class `error` and `complete` notifications: a stream can end abnormally (propagating one error to subscribers) or normally (no further values, but subscribers are told it's done). Signals have neither concept — a signal simply holds whatever its last `.set()` value was, forever, until the app or its owner is destroyed. When you `toSignal()` an observable that later errors, there's no signal-level "error state" to render; the error re-throws on the next read of the signal (e.g., inside a template binding or `computed()`),

which can crash unrelated rendering paths if not anticipated. The mitigation is to `catchError` upstream of `toSignal()` so the signal always holds a valid fallback value instead of ever propagating a throw.

Q13. Is `computed()` lazy or eager, and how does that compare to a cold vs hot observable?
`computed()` is lazy and memoized: it doesn't recompute when its dependencies change; it only marks itself dirty, and the actual recomputation happens the next time something *reads* it. This is somewhat analogous to a cold observable in that "work" only happens on demand — but unlike a cold observable (which re-executes its whole producer function per subscription), a `computed()`'s memoized result is shared across every consumer that reads it, more like a hot, multicast, `shareReplay(1)`-style observable in terms of "one computation, many readers," while still being lazy like a cold one in terms of "no computation until read."

Q14. Give an example of "glitch" behavior you might see with `combineLatest` that signals avoid entirely.

```
combineLatest([price$, quantity$]).pipe(
  map(([price, qty]) => price * qty)
)
```

If `price$` and `qty$` both emit new values "simultaneously" (e.g., both driven by a shared upstream event that maps into two separate streams), `combineLatest` may fire the projection function once per incoming emission rather than once for the "final settled" pair — you can observe a transient product computed from a stale `qty` and new `price`, then immediately another emission with both new. With `computed(() => price() * qty())`, Angular's dirty-marking ensures you only ever read the fully-settled combination — there's no transient intermediate emission because reads are pull-based and lazy, not pushed eagerly through each dependency change individually.
Interviewer intent: verifies the candidate can produce a concrete, technically accurate example of the abstract "glitch-free" claim rather than reciting the term without understanding it.

Q15. What's a realistic migration strategy for an existing large Angular app moving from RxJS-heavy state management (e.g., `BehaviorSubject`s in services) toward signals?
Migrate incrementally, service by service: (1) keep existing observable-based APIs at the service boundary for now if many consumers depend on them; (2) internally, replace `BehaviorSubject` state holders with `signal()`/`computed()` where the state is purely synchronous derived data, exposing a `toObservable()`-wrapped view only where external consumers still need streams; (3) for components, replace `| async` template bindings with direct signal reads as you touch each component, using `toSignal()` at the point where you consume a still-observable-based service API; (4) leave true async-orchestration logic (HTTP calls, debounced search, retries, WebSocket handling) in RxJS — don't force those into signals/effects; (5) use `takeUntilDestroyed()` to clean up any observables that must remain in components during the transition, since manual `ngOnDestroy` unsubscription is being phased out. The end state isn't "no RxJS" — it's RxJS confined to genuine async/streaming boundaries, with signals owning synchronous state and template-facing derivations.

---

7. Quick Revision Cheat Sheet

- **Signals:** pull-based, sync, single current value, glitch-free, no error/complete channel, no cancellation concept, no timing operators, auto teardown (no subscription to leak), ideal for derived UI state (`computed()`) and side effects (`effect()`).
- **RxJS:** push-based, sync or async, 0..N values over time, first-class `error`/`complete`, first-class

cancellation (`unsubscribe`, `switchMap`), rich timing/orchestration operators (`debounceTime`, `retry`, `combineLatest`, etc.), needs explicit teardown (`| async`, `takeUntilDestroyed()`).

- `toSignal(obs, { initialValue?, requireSync? })`: observable → signal; needs injection context (or `{ injector }`); subscribes eagerly; auto-unsubscribes via `DestroyRef`; no `initialValue` ⇒ type includes `undefined`; errors re-throw on next read unless caught upstream.
- `toObservable(sig, { injector? })`: signal → observable; internally an `effect()` that emits on each (batched, scheduled) change; emits current value immediately on subscribe (BehaviorSubject-like); can coalesce rapid `.set()` calls into a single emission — not a 1:1 event log.
- **Typeahead = RxJS's home turf:** `debounceTime` + `distinctUntilChanged` + `switchMap` + `catchError` in a few lines; reimplementing with signals+`effect()` means hand-rolled timers, cancellation tokens, and cleanup — more code, more bugs, no composability.
- **Cart totals / simple derived state = signals' home turf:** `computed()` chains, no subscriptions, no `async` pipe, trivially synchronous unit tests.
- **Golden rule:** signals own *state*, RxJS owns *process*. Bridge at the boundary with `toSignal()` / `toObservable()`; don't fake one system's strengths inside the other.
- **Common gotchas:** missing injection context for `toSignal` / `toObservable`; forgetting `initialValue`; assuming `toObservable` emissions are synchronous with `.set()`; assuming every `.set()` produces a distinct emission; writing to a signal an effect also reads (infinite-loop guard); letting `toSignal()`'d errors propagate uncaught into template reads.

Created By - Durgesh Singh